

CISO Assistant

Gestionnaire générique d'objets API

Documentation utilisateur — Version 3.2

Généré le 13 avril 2026

1. Présentation

`ciso\_manager.py` est un outil en ligne de commande Python permettant de gérer n'importe quelle ressource exposée par l'API REST de CISO Assistant.

Il couvre cinq opérations principales :

- **export** — exporter une ressource en JSON ré-importable
- **import** — créer ou mettre à jour des objets depuis un fichier JSON
- **get** — afficher une ressource dans le terminal
- **show** — afficher le JSON brut d'un objet par son UUID
- **folders** — lister les dossiers et leurs UUIDs

1.1 Prérequis

- Python 3.7 ou supérieur (aucune dépendance externe — bibliothèque standard uniquement)
- Un Personal Access Token CISO Assistant (Profil → API Tokens → Créer)
- Accès réseau à l'instance CISO Assistant

2. Configuration

2.1 Variables d'environnement

L'outil se configure exclusivement via des variables d'environnement. Aucun fichier de configuration n'est nécessaire.

Variable	Défaut	Description
CISO_API_TOKEN	(obligatoire)	Personal Access Token
CISO_BASE_URL	https://localhost:8443/api	URL de base de l'API
CISO_VERIFY_SSL	false	Vérification du certificat SSL (true/false)
CISO_ENUM_CASE	lower	Casse des enums à l'export (none/lower/upper)

2.2 Définir les variables

Windows (PowerShell)

```
$env:CISO_API_TOKEN = "votre_token"
$env:CISO_BASE_URL = "https://ciso.mondomaine.com/api"
$env:CISO_VERIFY_SSL = "true"
```

Linux / macOS (bash)

```
export CISO_API_TOKEN="votre_token"
export CISO_BASE_URL="https://ciso.mondomaine.com/api"
export CISO_VERIFY_SSL="true"
```

Astuce : placez ces exports dans votre .bashrc / profil PowerShell pour les rendre permanents.

3. Référence des commandes

3.1 export

Récupère l'intégralité d'une ressource (toutes pages) et l'enregistre dans un fichier JSON aplati, directement ré-importable.

```
python ciso_manager.py export <ressource> [options]
```

Commande / Option	Description
<ressource>	Nom de la ressource ou chemin relatif (ex : applied-controls, risk-scenarios)
--output, -o	Fichier de sortie (défaut : <ressource>_YYYYMMDD_HHMMSS.json)
--fields, -c	Champs à conserver séparés par des virgules — le champ id est toujours inclus
--param CLE=VAL	Filtre côté API (répétable) — transmis directement en query string
--where EXPR	Filtre local post-récupération (répétable) — voir section 4
--enum-case	Normalise la casse de certains enums : none (défaut) lower upper

Exemples

```
python ciso_manager.py export applied-controls --output controls.json
python ciso_manager.py export applied-controls --param status=active --output actifs.json
python ciso_manager.py export applied-controls --where status=active --output actifs_local.json
python ciso_manager.py export metrics --fields id,name,status --output metrics_light.json
```

3.2 import

Crée ou met à jour des objets depuis un fichier JSON. Le comportement dépend de la présence d'un identifiant dans chaque objet.

```
python ciso_manager.py import <ressource> --file <fichier.json> [options]
```

Commande / Option	Description
<ressource>	Ressource cible (ex : metric-instances, applied-controls)
--file, -f	Chemin vers le fichier JSON à importer (obligatoire)
--key, -k	Clé d'identification : id (défaut) ou ref_id
--dry-run	Simule l'import sans envoyer de requêtes modifiantes — affiche les payloads
--exclude, -e	Champs à ignorer lors de l'import, séparés par des virgules

Logique de création / mise à jour

Le comportement dépend de l'option --key :

--key	Identifiant trouvé ?	Action
-------	----------------------	--------

id	Oui (UUID présent)	PATCH sur /<ressource>/<uuid>/
id	Non (pas d'id)	POST — création d'un nouvel objet
ref_id	ref_id résolu	UUID résolu via index GET, puis PATCH
ref_id	ref_id introuvable	POST — création d'un nouvel objet

Avec `--key ref_id`, le script effectue un GET complet sur la ressource au démarrage pour construire l'index `ref_id` → UUID avant tout PATCH.

Formats JSON acceptés

- **Liste** : [{"id": "...", "name": "..."}, ...]
- **Objet paginé** : {"results": [...]}
- **Objet unique** : {"id": "...", "name": "..."}]

Exemples

```
python ciso_manager.py import applied-controls --file controls.json
python ciso_manager.py import applied-controls --file controls.json --key ref_id
python ciso_manager.py import applied-controls --file controls.json --key ref_id --dry-run
python ciso_manager.py import applied-controls --file controls.json --exclude folder,category
```

3.3 get

Affiche une ressource dans le terminal avec pagination et filtres.

```
python ciso_manager.py get <ressource> [options]
```

Commande / Option	Description
<ressource>	Ressource ou URL complète
--page, -p	Numéro de page à afficher
--page-size, -n	Nombre d'éléments par page
--status, -s	Filtre sur le champ status
--search, -q	Recherche textuelle
--fields, -c	Colonnes à afficher, séparées par des virgules
--format, -f	Format d'affichage : table (défaut) json simple
--all, -a	Récupère et affiche toutes les pages en une seule fois
--param CLE=VAL	Paramètres API supplémentaires (répétable)

Exemples

```
python ciso_manager.py get applied-controls
python ciso_manager.py get applied-controls --status active --format simple
python ciso_manager.py get applied-controls --all --fields id,name,status
python ciso_manager.py get risk-scenarios --page 2 --page-size 20
```

3.4 show

Affiche le JSON brut et complet d'un objet identifié par son UUID.

```
python ciso_manager.py show <ressource> --id <uuid>
```

Commande / Option	Description
<ressource>	Ressource cible
--id	UUID de l'objet à afficher (obligatoire)

Exemple

```
python ciso_manager.py show applied-controls --id abc-123-...
```

3.5 folders

Liste tous les dossiers de l'instance avec leur UUID et leur dossier parent. Utile pour récupérer les UUIDs à utiliser dans les imports.

```
python ciso_manager.py folders
```

4. Filtres locaux (--where)

L'option `--where` permet de filtrer les résultats localement, après récupération complète depuis l'API. Elle est disponible sur la commande `export` et s'applique sur n'importe quel champ, y compris les champs imbriqués via la notation pointée.

```
--where CHAMP<opérateur>VALEUR
```

Opérateur	Syntaxe	Exemple
<code>=</code> ou <code>==</code>	<code>champ=valeur</code>	<code>--where status=active</code>
<code>!=</code>	<code>champ!=valeur</code>	<code>--where status!=obsolete</code>
<code>~</code>	<code>champ~sous-chaine</code>	<code>--where name~backup</code>
<code>in</code>	<code>champ in val1,val2</code>	<code>--where status in active,in_progress</code>
<code>></code>	<code>champ>nombre</code>	<code>--where effort>2</code>
<code>>=</code>	<code>champ>=nombre</code>	<code>--where priority>=3</code>
<code><</code>	<code>champ<nombre</code>	<code>--where effort<5</code>
<code><=</code>	<code>champ<=nombre</code>	<code>--where priority<=2</code>

L'option `--where` est répétable. Plusieurs clauses sont combinées en AND implicite.

Notation imbriquée

Accès aux champs imbriqués via la notation pointée :

```
--where folder.name=MonDossier
--where perimeter.id=uuid-xxx
```

Exemples combinés

```
python ciso_manager.py export applied-controls \
  --where status=active --where effort>=2 --output result.json
```

5. Ressources disponibles

Le tableau suivant liste tous les raccourcis acceptés par l'outil et leur chemin API correspondant.

Raccourci	Chemin API
Gouvernance	
policies	policies
applied-controls	applied-controls
reference-controls	reference-controls
filtering-labels	filtering-labels
folders	folders
perimeters	perimeters
Risques	
risk-assessments	risk-assessments
risk-scenarios	risk-scenarios
risk-acceptances	risk-acceptances
risk-matrices	risk-matrices
threats	threats
vulnerabilities	vulnerabilities
Conformité	
compliance-assessments	compliance-assessments
requirement-assessments	requirement-assessments
frameworks	frameworks
Actifs	
assets	assets
asset-class	asset-class
asset-capabilities	asset-capabilities
Métrologie	
metric-definitions	metrology/metric-definitions
metric-instances	metrology/metric-instances
custom-metric-samples	metrology/custom-metric-samples
dashboards	metrology/dashboards
dashboard-widgets	metrology/dashboard-widgets
Entités & Tiers	
entities	entities
entity-assessments	entity-assessments
representatives	representatives
contracts	contracts
solutions	solutions
Preuves & Tâches	
evidences	evidences

task-templates	task-templates
task-nodes	task-nodes
campaigns	campaigns
Findings & Exceptions	
findings	findings
findings-assessments	findings-assessments
security-exceptions	security-exceptions
Incidents	
incidents	incidents
Résilience	
asset-assessments	resilience/asset-assessments
business-impact-analysis	resilience/business-impact-analysis
EBIOS-RM	
ebios-studies	ebios-rm/studies
ebios-feared-events	ebios-rm/feared-events
ebios-stakeholders	ebios-rm/stakeholders
ebios-attack-paths	ebios-rm/attack-paths
ebios-operational-scenarios	ebios-rm/operational-scenarios
CRQ	
crq-studies	crq/quantitative-risk-studies
crq-scenarios	crq/quantitative-risk-scenarios
Privacy	
processings	privacy/processings
personal-data	privacy/personal-data
data-breaches	privacy/data-breaches
right-requests	privacy/right-requests
IAM & Paramètres	
users	users
user-groups	user-groups
role-assignments	role-assignments
teams	teams
Divers	
actors	actors
webhooks	webhooks/endpoints
loaded-libraries	loaded-libraries
stored-libraries	stored-libraries

Vous pouvez aussi passer directement n'importe quel chemin relatif (ex : `privacy/processings`) ou une URL complète à la place d'un raccourci.

6. Cas d'usage typiques

6.1 Export → modification → ré-import

Workflow standard pour mettre à jour en masse des objets existants.

```
# 1. Exporter avec les champs utiles
python ciso_manager.py export applied-controls \
  --fields id,ref_id,name,status --output controls.json
```

```
# 2. Éditer controls.json dans un éditeur
```

```
# 3. Ré-importer par UUID (défaut)
python ciso_manager.py import applied-controls --file controls.json
```

6.2 Import multi-environnements via ref_id

Utile lorsque les UUIDs diffèrent entre environnements (dev / prod) mais que les ref_id sont stables.

```
# Export depuis prod (ref_id sera dans le fichier)
python ciso_manager.py export applied-controls --output controls.json
```

```
# Import sur dev – le script résout les UUIDs automatiquement
python ciso_manager.py import applied-controls \
  --file controls.json --key ref_id
```

6.3 Simuler avant d'appliquer

```
python ciso_manager.py import metric-instances \
  --file metrics.json --key ref_id --dry-run
```

Le dry-run affiche les payloads et les URLs cibles sans effectuer aucune requête modifiante (POST / PATCH).

6.4 Créer de nouveaux objets en masse

Un fichier JSON sans champ id (ni ref_id connu) provoque systématiquement un POST.

```
# fichier new_controls.json
[
  { "name": "Nouveau contrôle 1", "status": "active" },
  { "name": "Nouveau contrôle 2", "status": "in_progress" }
]
```

```
python ciso_manager.py import applied-controls --file new_controls.json
```

6.5 Éviter les problèmes de cache Python (Windows)

Sur Windows, Python peut utiliser des fichiers .pyc en cache et ignorer une mise à jour du script.

```
# Supprimer le cache
Remove-Item -Recurse -Force __pycache__
```

```
# Ou lancer avec -B pour désactiver le cache
python.exe -B ciso_manager.py import ...
```


7. Codes de retour et messages

7.1 Codes de sortie

Code	Signification
0	Succès — aucune erreur
1	Au moins une erreur lors de l'import, ou erreur fatale

7.2 Préfixes des messages

Préfixe	Signification
[INFO]	Message informatif — déroulement normal
[OK]	Opération réussie
[ERREUR]	Erreur bloquante — l'opération est interrompue
[ERREUR N]	Réponse HTTP en erreur avec le code N
[AVERTISSEMENT]	Anomalie non bloquante (ex : paramètre ignoré)
[DRY-RUN]	Mode simulation — aucune modification effectuée
✓	Objet créé ou mis à jour avec succès
✗	Erreur sur cet objet spécifique

